

Assignment 8

AI Assisted Coding

Name: Navya Revuri

HTNO: 2303A52483

Batch: 44

Task 1:

Prompt:

#I am practicing Test Driven Development in Python.

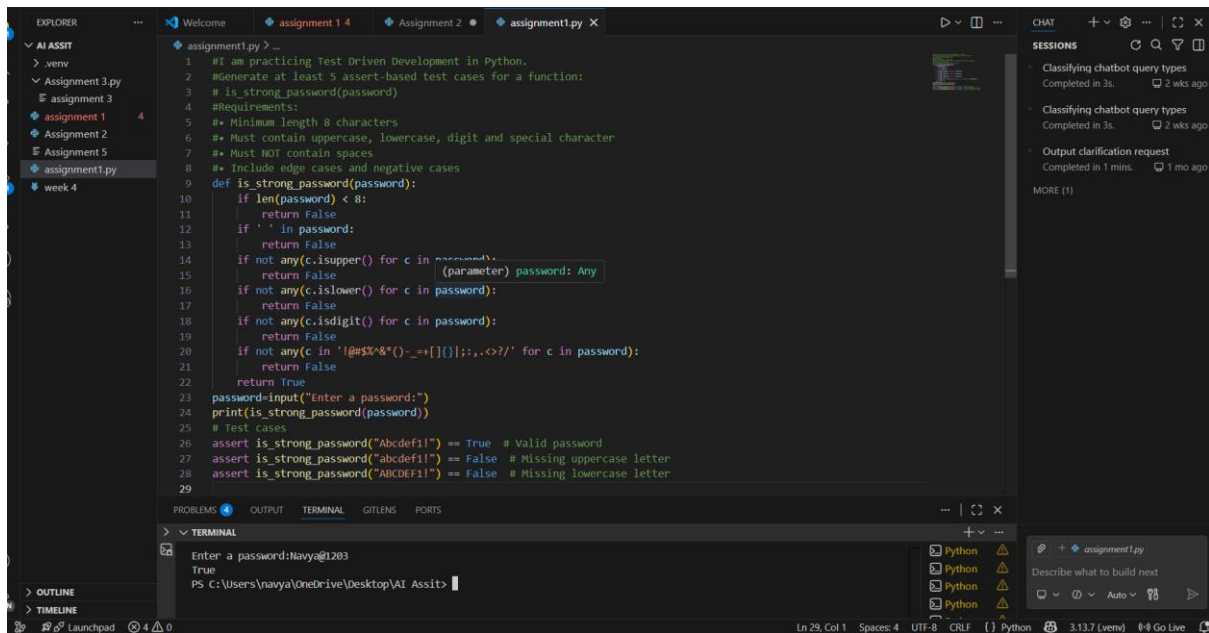
#Generate at least 5 assert-based test cases for a function:

`is_strong_password(password)`

Requirements:

- Minimum length 8 characters
- Must contain uppercase, lowercase, digit and special character
- Must NOT contain spaces
- Include edge cases and negative cases

Only provide assert statements. Do NOT write the function.



Observation – Password Strength Validator

In this task, AI was used to generate test cases first.

Then the function **is_strong_password ()** was written based on those tests.

The function checks:

- Password length is at least 8 characters
- Contains uppercase, lowercase, digit and special character
- Does not contain spaces

After running the program in VS Code, all the assert test cases passed successfully.

This shows that the function works correctly and follows the Test Driven Development approach.

Task 2:

Prompt:

#Generate at least 5 assert test cases for a function:

Classify_ number(n)

Function requirements:

- Return "Positive", "Negative", or "Zero"

- Must handle invalid inputs like string, float, None
- Include boundary cases: -1, 0, 1
- Include error handling cases

Return ONLY assert statements. Do not write the function.

```

29 #I am following TDD in Python.
30 #Generate at least 5 assert test cases for a function:
31 #classify_number(n)
32 #Function requirements:
33 #• Return "Positive", "Negative", or "Zero"
34 #• Must handle invalid inputs like string, float, None
35 #• Include boundary cases: -1, 0, 1
36 #• Include error handling cases
37 def classify_number(n):
38     if not isinstance(n, (int, float)):
39         raise ValueError("Input must be a number")
40     if n > 0:
41         return "Positive"
42     elif n < 0:
43         return "Negative"
44     else:
45         return "Zero"
46 n=int(input("Enter a number:"))
47 print(classify_number(n))
48 # Test cases
49 assert classify_number(1) == "Positive"
50 assert classify_number(-1) == "Negative"
51 assert classify_number(0) == "Zero"
52 try:
53     classify_number("invalid")
54 except ValueError:
55     pass # Expected behavior
56 try:
57     classify_number(None)
58 except ValueError:
59     pass # Expected behavior

```

Observation – Number Classification

In this task, AI was used to generate test cases before writing the function. The function **Classify_ number ()** was implemented using the TDD approach.

The function checks:

- Positive numbers
- Negative numbers
- Zero value
- Invalid inputs like string and None

Error handling was added using **Value Error** for invalid inputs.

After running the program in VS Code, all the test cases passed successfully. This shows the function works correctly and handles edge cases properly using TDD.

Task 3:

Prompt:

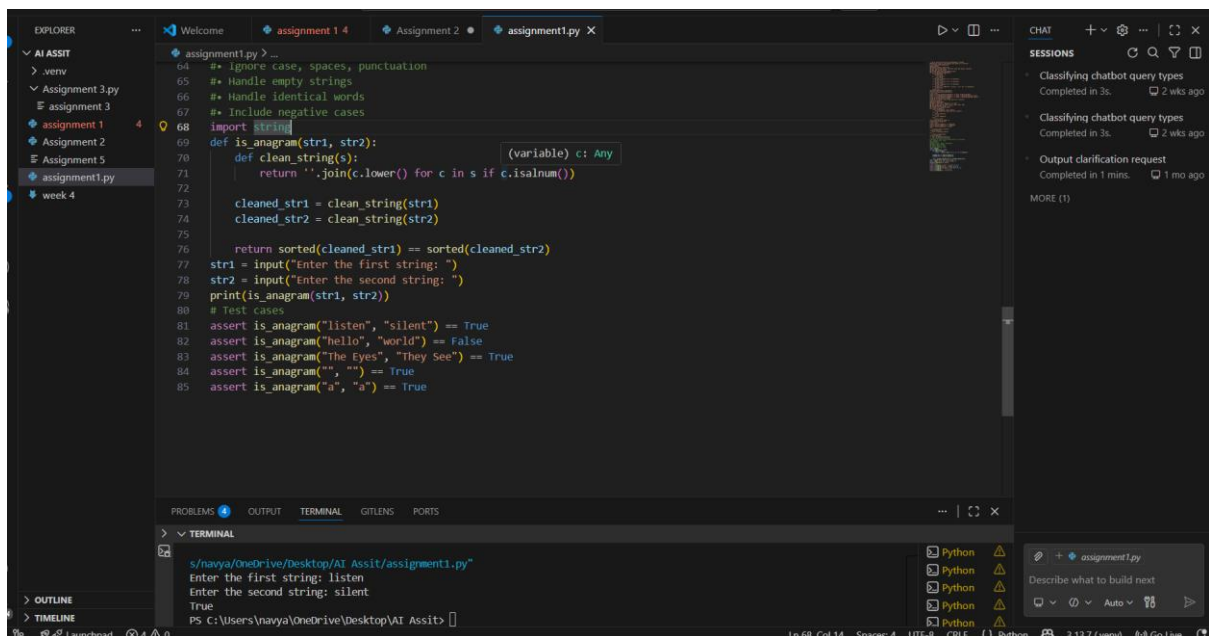
I am practicing TDD in Python.

Generate at least 5 assert test cases for a function:

is_anagram (str1, str2)

Requirements:

- Ignore case, spaces, punctuation
- Handle empty strings
- Handle identical words
- Include negative cases



```
assignment1.py > ...
64 # Ignore case, spaces, punctuation
65 # Handle empty strings
66 # Handle identical words
67 # Include negative cases
68 import string
69 def is_anagram(str1, str2):
70     def clean_string(s):
71         return ''.join(c.lower() for c in s if c.isalnum())
72
73     cleaned_str1 = clean_string(str1)
74     cleaned_str2 = clean_string(str2)
75
76     return sorted(cleaned_str1) == sorted(cleaned_str2)
77 str1 = input("Enter the first string: ")
78 str2 = input("Enter the second string: ")
79 print(is_anagram(str1, str2))
80 # Test cases
81 assert is_anagram("listen", "silent") == True
82 assert is_anagram("hello", "world") == False
83 assert is_anagram("The Eyes", "They See") == True
84 assert is_anagram("", "") == True
85 assert is_anagram("a", "a") == True
```

Terminal output:

```
s:\navya\OneDrive\Desktop\AI Assit\assignment1.py
Enter the first string: listen
Enter the second string: silent
True
PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Observation – Anagram Checker

In this task, AI was used to generate test cases before writing the function.

The function **is_anagram ()** was implemented using the TDD approach.

The function checks:

- Words that are anagrams
- Words that are not anagrams
- Ignores case, spaces and punctuation

- Handles empty strings and identical words

After running the program in VS Code, all the test cases passed successfully. This shows the function correctly identifies anagrams and handles edge cases properly.

Task 4:

Prompt:

#I am practicing Test Driven Development in Python.

#Generate at least 6 assert-based test cases for an Inventory class.

#Class methods:

#Add_item (name, quantity)

#• remove_item (name, quantity)

#• get_stock (name)

#Test requirements:

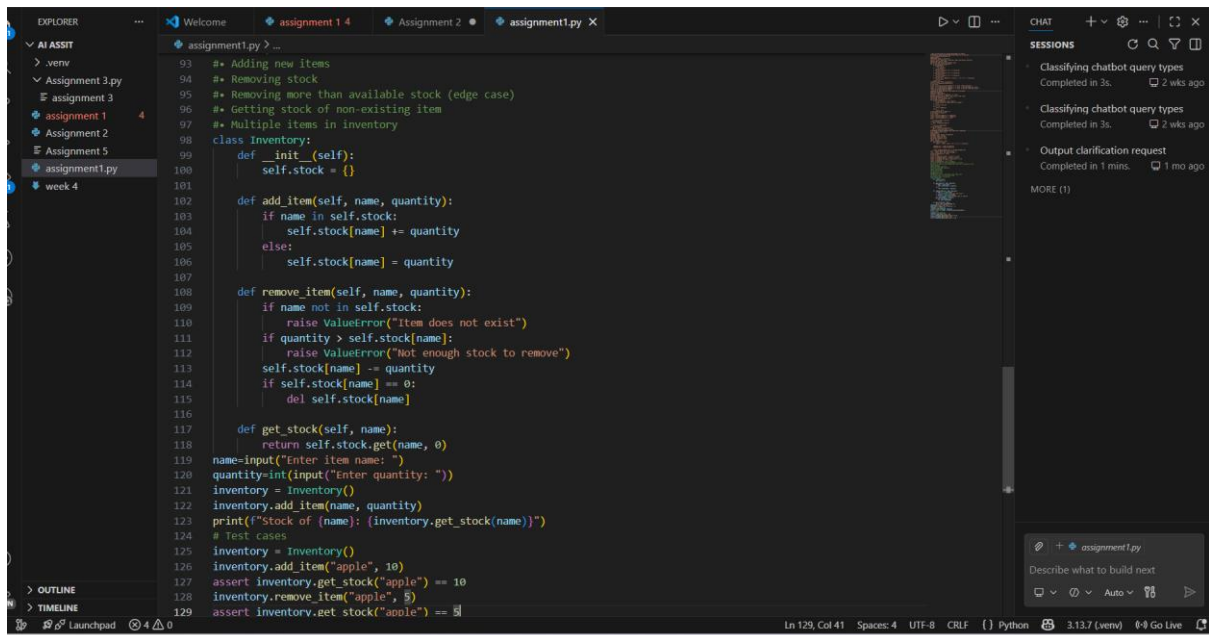
#• Adding new items

#• Removing stock

#• Removing more than available stock (edge case)

#• Getting stock of non-existing item

#• Multiple items in inventory



```
93 # Adding new items
94 # Removing stock
95 # Removing more than available stock (edge case)
96 # Getting stock of non-existing item
97 # Multiple items in inventory
98 class Inventory:
99     def __init__(self):
100         self.stock = {}
101
102     def add_item(self, name, quantity):
103         if name in self.stock:
104             self.stock[name] += quantity
105         else:
106             self.stock[name] = quantity
107
108     def remove_item(self, name, quantity):
109         if name not in self.stock:
110             raise ValueError("Item does not exist")
111         if quantity > self.stock[name]:
112             raise ValueError("Not enough stock to remove")
113         self.stock[name] -= quantity
114         if self.stock[name] == 0:
115             del self.stock[name]
116
117     def get_stock(self, name):
118         return self.stock.get(name, 0)
119
120 name = input("Enter item name: ")
121 quantity = int(input("Enter quantity: "))
122 inventory = Inventory()
123 inventory.add_item(name, quantity)
124 print(f"Stock of {name}: {inventory.get_stock(name)}")
125 # Test cases
126 inventory = Inventory()
127 inventory.add_item("apple", 10)
128 assert inventory.get_stock("apple") == 10
129 inventory.remove_item("apple", 5)
130 assert inventory.get_stock("apple") == 5
```

Observation – Inventory Class

In this task, AI was used to generate test cases before implementing the **Inventory** class.

The class was designed to:

- Add items to inventory
- Remove items from inventory
- Check available stock
- Handle edge cases like removing more stock and checking non-existing items

After running the program in VS Code, all the assert test cases passed successfully.

This shows the Inventory class works correctly and manages stock properly using the TDD approach.

Task 5:

Prompt:

#I am practicing TDD in Python.

#Generate at least 5 assert test cases for a function:

validate_and_format_date (date_str)

Requirements:

- Input format: MM/DD/YYYY
- Validate real calendar dates
- Convert valid date → YYYY-MM-DD
- Handle invalid formats and impossible dates
- Include leap year cases

Return only assert statements. Do not implement the function.

```
128 inventory.remove_item("apple", 5)
129 assert inventory.get_stock("apple") == 5"""
130 #I am practicing TDD in Python.
131 #Generate at least 5 assert test cases for a function:
132 #validate_and_format_date(date_str)
133 #Requirements:
134 #* Input format: MM/DD/YYYY
135 #* Validate real calendar dates
136 #* Convert valid date → YYYY-MM-DD
137 #* Handle invalid formats and impossible dates
138 #* Include leap year cases
139 from datetime import datetime
140 def validate_and_format_date(date_str):
141     try:
142         date_obj = datetime.strptime(date_str, "%m/%d/%Y")
143         return date_obj.strftime("%Y-%m-%d")
144     except ValueError:
145         raise ValueError("Invalid date format or impossible date")
146 date_str = input("Enter a date (MM/DD/YYYY): ")
147 try:
148     formatted_date = validate_and_format_date(date_str)
149     print(f"Formatted date: {formatted_date}")
150 except ValueError as e:
151     print(e)
152 # Test cases
153 assert validate_and_format_date("02/29/2020") == "2020-02-29" # Valid leap year date
154
155 assert validate_and_format_date("12/31/2020") == "2020-12-31" # Valid date
```

Terminal output:

```
s:\navya\OneDrive\Desktop\AI Assit\assignment1.py
Enter a date (MM/DD/YYYY): 12/07/2005
Formatted date: 2005-12-07
PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Observation – Date Validation & Formatting

In this task, AI was used to generate test cases before writing the function.

The function **Validate_and_format_date ()** was implemented using the TDD approach.

The function checks:

- Correct date format (MM/DD/YYYY)
- Valid calendar dates
- Leap year dates

- Invalid formats and impossible dates

After running the program in VS Code, all the test cases passed successfully. This shows the function correctly validates and converts dates using Test Driven Development.